Первоначальная настройка git

Цель работы

- Изучить идеологию и применение средств контроля версий.
- Освоить умения по работе с git.

Задание

- Создать базовую конфигурацию для работы с git.
- Создать ключ SSH.
- Создать ключ PGP.
- Настроить подписи git.
- Зарегистрироваться на Github.
- Создать локальный каталог для выполнения заданий по предмету.

Теоретическое введение

Системы контроля версий. Общие понятия

Системы контроля версий (Version Control System, VCS) применяются при работе нескольких человек над одним проектом. Обычно основное дерево проекта хранится в локальном или удалённом репозитории, к которому настроен доступ для участников проекта. При внесении изменений в содержание проекта система контроля версий позволяет их фиксировать, совмещать изменения, произведённые разными участниками проекта, производить откат к любой более ранней версии проекта, если это требуется.

В классических системах контроля версий используется централизованная модель, предполагающая наличие единого репозитория для хранения файлов. Выполнение большинства функций по управлению версиями осуществляется специальным сервером. Участник проекта (пользователь) перед началом работы посредством определённых команд получает нужную ему версию файлов. После внесения изменений, пользователь размещает новую версию в хранилище. При этом предыдущие версии не удаляются из центрального хранилища и к ним можно вернуться в любой момент. Сервер может сохранять не полную версию изменённых файлов, а производить так называемую дельта-компрессию — сохранять только изменения между последовательными версиями, что позволяет уменьшить объём хранимых данных.

Системы контроля версий поддерживают возможность отслеживания и разрешения конфликтов, которые могут возникнуть при работе нескольких человек над одним файлом. Можно объединить (слить) изменения, сделанные разными участниками (автоматически или вручную), вручную выбрать нужную версию, отменить изменения вовсе или заблокировать файлы для изменения. В зависимости от настроек блокировка не позволяет другим пользователям получить рабочую копию или препятствует изменению рабочей копии файла средствами файловой системы ОС, обеспечивая таким образом, привилегированный доступ только одному пользователю, работающему с файлом.

Системы контроля версий также могут обеспечивать дополнительные, более гибкие функциональные возможности. Например, они могут поддерживать работу с несколькими версиями одного файла, сохраняя общую историю изменений до точки ветвления версий и собственные истории изменений каждой ветви. Кроме того, обычно доступна информация о том, кто из участников, когда и какие изменения вносил. Обычно такого рода информация хранится в журнале изменений, доступ к которому можно ограничить.

В отличие от классических, в распределённых системах контроля версий центральный репозиторий не является обязательным.

Среди классических VCS наиболее известны CVS, Subversion, а среди распределённых — Git, Bazaar, Mercurial. Принципы их работы схожи, отличаются они в основном синтаксисом используемых в работе команд.

Примеры использования git

Система контроля версий Git представляет собой набор программ командной строки. Доступ к ним можно получить из терминала посредством ввода команды git с различными опциями. Благодаря тому, что Git является распределённой системой контроля версий, резервную копию локального хранилища можно сделать простым копированием или архивацией.

Основные команды git

Перечислим наиболее часто используемые команды git.

Создание основного дерева репозитория:

```
git init
```

Получение обновлений (изменений) текущего дерева из центрального репозитория:

```
git pull
```

Отправка всех произведённых изменений локального дерева в центральный репозиторий:

```
git push
```

Просмотр списка изменённых файлов в текущей директории:

```
git status
```

Просмотр текущих изменений:

```
git diff
```

Сохранение текущих изменений:

```
  добавить все изменённые и/или созданные файлы и/или каталоги:

    git add .

  добавить конкретные изменённые и/или созданные файлы и/или
каталоги:

    git add имена_файлов

  удалить файл и/или каталог из индекса репозитория (при этом файл
и/или каталог остаётся в локальной директории):

    git rm имена_файлов

Сохранение добавленных изменений:

  сохранить все добавленные изменения и все изменённые файлы:

    git commit -am 'Описание коммита'

  сохранить добавленные изменения с внесением комментария через
встроенный редактор:

    git commit

  создание новой ветки, базирующейся на текущей:

    git checkout -b имя_ветки

  переключение на некоторую ветку:

    git checkout имя_ветки

    (при переключении на ветку, которой ещё нет в локальном
репозитории, она будет создана и связана с удалённой)

    отправка изменений конкретной ветки в центральный репозиторий:
```

```
    git push origin имя_ветки

   слияние ветки с текущим деревом:

    git merge --no-ff имя_ветки

Удаление ветки:

    удаление локальной уже слитой с основным деревом ветки:

    git branch -d имя_ветки

    принудительное удаление локальной ветки:

    git branch -D имя_ветки

    удаление ветки с центрального репозитория:

    git push origin :имя_ветки
```

Стандартные процедуры работы при наличии центрального репозитория

Работа пользователя со своей веткой начинается с проверки и получения изменений из центрального репозитория (при этом в локальное дерево до начала этой процедуры не должно было вноситься изменений):

```
git checkout master
git pull
git checkout -b имя_ветки
```

Затем можно вносить изменения в локальном дереве и/или ветке.

После завершения внесения какого-то изменения в файлы и/или каталоги проекта необходимо разместить их в центральном репозитории. Для этого необходимо проверить, какие файлы изменились к текущему моменту:

```
git status
```

При необходимости удаляем лишние файлы, которые не хотим отправлять в центральный репозиторий.

Затем полезно просмотреть текст изменений на предмет соответствия правилам ведения чистых коммитов:

```
git diff
```

Если какие-либо файлы не должны попасть в коммит, то помечаем только те файлы, изменения которых нужно сохранить. Для этого используем команды добавления и/или удаления с нужными

опциями:

```
git add …
git rm …
```

Если нужно сохранить все изменения в текущем каталоге, то используем:

```
git add .
```

Затем сохраняем изменения, поясняя, что было сделано:

```
git commit -am "Some commit message"
```

```
Отправляем изменения в центральный репозиторий:
```

```
git push origin имя_ветки
```

```
или
```

```
git push
```

Выполнение лабораторной работы

Базовая настройка git

```
kmborisova@fedora:~$ git config --global user.name "Имя Фамилия"~
\kmborisova@fedora:~$git config --global user.name "kmborisova"
kmborisova@fedora:~$ git config --global  user.email "xeniaborisova634@gmail.com"
kmborisova@fedora:~$ git config --global quotepath false
error: key does not contain a section: quotepath
kmborisova@fedora:~$ git config --global core.quotepath false
kmborisova@fedora:~$ git config --global init.defaultBranch master
kmborisova@fedora:~$ git config --global core.autocrlf input
kmborisova@fedora:~$ git config --global core.safecrl warn
```

рис 1

Создание SSH-ключей

```
kmborisova@fedora:~/tutorial$ ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/home/kmborisova/.ssh/id_rsa): keys.txt
Enter passphrase for "keys.txt" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in keys.txt
Your public key has been saved in keys.txt.pub
The key fingerprint is:
SHA256:P3Ssx9biyhndVqarFtfq+w7zG00fMnzfBZMVAk6hoAM kmborisova@fedora
The key's randomart image is:
+---[RSA 4096]----+
|   E   .   +o. .o|
|    . . . +   .o |
|     o   . .  +  |
|      .    ..  o |
|        S . o+ +=|
|         o =.o==O|
|          = *oBo=|
|         . B.o.=.|
|          +ooo+==|
+----[SHA256]-----+
kmborisova@fedora:~/tutorial$ ssh-keygen -t ed25519
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/kmborisova/.ssh/id_ed25519): keyed.txt
Enter passphrase for "keyed.txt" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in keyed.txt
Your public key has been saved in keyed.txt.pub
The key fingerprint is:
SHA256:n4TH2AQpDf30BNeHajgFd+7FE1mWmBHxBeC/2Bc7YI4 kmborisova@fedora
The key's randomart image is:
+--[ED25519 256]--+
|      .+..oo+*O+B|
|      . +..+o=+=+|
|       . o.=...+o|
|         *+ +o ..|
|        S =o oo. |
|         + .+o..o|
|          oE..oo.|
|               ..|
|                 |
+----[SHA256]-----+
```

рис 2

Создание PGP-ключа

```
kmborisova@fedora:~/tutorial$ gpg --full-generate-key
gpg (GnuPG) 2.4.8; Copyright (C) 2025 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

gpg: создан каталог '/home/kmborisova/.gnupg'
Выберите тип ключа:
   (1) RSA and RSA
   (2) DSA and Elgamal
   (3) DSA (sign only)
   (4) RSA (sign only)
   (9) ECC (sign and encrypt) *default*
  (10) ECC (только для подписи)
  (14) Existing key from card
Ваш выбор? 1
длина ключей RSA может быть от 1024 до 4096.
Какой размер ключа Вам необходим? (3072) 4096
Запрошенный размер ключа - 4096 бит
Выберите срок действия ключа.
         0 = не ограничен
      <n>  = срок действия ключа - n дней
      <n>w = срок действия ключа - n недель
      <n>m = срок действия ключа - n месяцев
      <n>y = срок действия ключа - n лет
Срок действия ключа? (0) 0
Срок действия ключа не ограничен
Все верно? (y/N) y

GnuPG должен составить идентификатор пользователя для идентификации ключа.

Ваше полное имя: kmborisova
Адрес электронной почты: xeniaborisova634@gmail.com
Примечание:
Вы выбрали следующий идентификатор пользователя:
    "kmborisova <xeniaborisova634@gmail.com>"
```

рис 3

Добавление SSH-ключей в GitHub

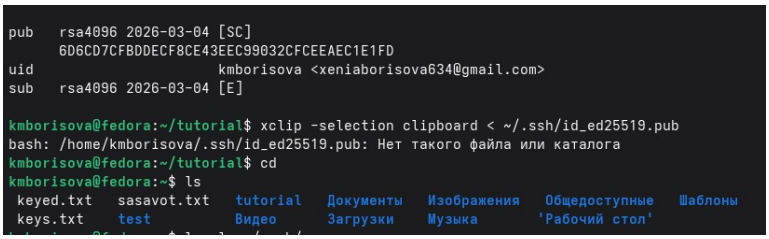

рис 4

Добавление PGP-ключа в GitHub

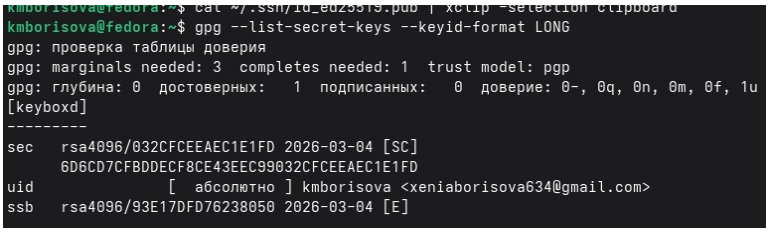

рис 5

Настройка автоматической подписи коммитов

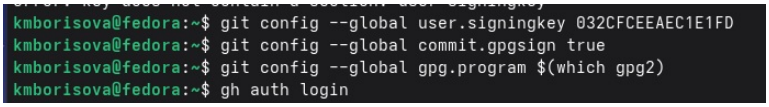

рис 6

Настройка GitHub CLI

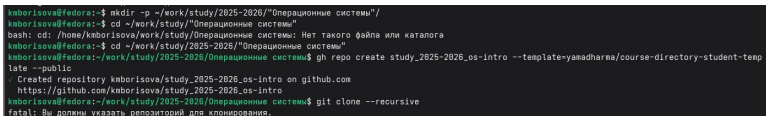

рис 7

Создание рабочего пространства

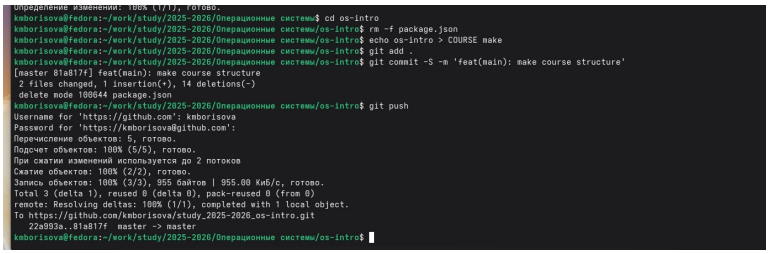

рис 8

Выводы

В результате выполнения лабораторной работы была настроена базовая конфигурация git , созданы SSH PGP ключи, выполнена их привязка на GitHub . Настроена автоматическая подпись коммитов. Создан репозиторий из шаблона, проведена его инициализация и отправка на GitHub.

Список литературы

ТУИС. Архитектура компьютеров и операционные системы. Раздел “Операционные системы”. Лабораторная работа №2.

https://esystem.rudn.ru/mod/page/view.php?id=1358324